Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

EcoRouting Engine: Hazard-Aware Path Optimization and Fill-Rate-Adaptive Waste Scheduling

Student/Team Information

Team Name: *Team #* *PBL Mentor:*	*EcoGraph* *DSCPP-III-2026-T186* *Mr. Navneet Rajput*
Team member 1 (Team Lead)	*Jain, Tejas* *Student ID: 2510011541* *Roll No. 2028228, Sec A* *aispace101@gmail.com* 
Team member 2	*Negi, Apoorv* *Student ID: 2510011088* *Roll No. 2027640, Sec D* *apoorvnegi17@gmail.com* 
Team member 3	*Nigam, Yash* *Student ID: 2510010441* *Roll No. 2028289, Sec H* *nigamyash07@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Anyone who has watched a municipal garbage truck on its route notices the same thing: it drives the same streets every day, in the same order, whether or not there is anything to collect. Routes are fixed by habit rather than the actual shortest path between the depot, the homes on the list, and the disposal bins, wasting fuel, driver time, and public money. Two problems compound this. Hazardous waste – medical sharps, expired chemicals, batteries – is often collected the same way as ordinary household waste, even though it should be picked up immediately and kept off residential and commercial streets on its way to disposal; delays or careless routing here are a genuine safety and environmental risk. Second, every bin is treated as if it fills at the same rate, so a fixed weekly schedule either visits a bin that is nowhere near full or leaves an overflowing one unattended. We wanted a system that treats the city as the graph it actually is, and routes, restricts, and schedules collection accordingly.

Our goal is a working C++ backend that models the city as a weighted, zone-tagged graph and answers three questions on demand: the shortest route to collect from a given set of homes, which locations are actually due today based on fill-rate, and how to route hazardous waste safely away from residential streets – all exposed through a REST API and shown live on a Cytoscape.js frontend that animates the computed route.

Updated Project Approach and Architecture

Design. One C++17 process serves the REST API and the front end on one port. `controllers.cpp` handles HTTP and JSON behind a mutex; domain classes (`Graph`, Dijkstra, `Scheduler`, classifier, route builders, `Simulation`) work on an in-memory adjacency-list city loaded from `city.json`.

Routing. `RouteStrategy` is an abstract base class; the Priority and FIFO subclasses set the stop order.

Protocol. REST over HTTP with JSON, 13 endpoints, one error format.

Front end. Plain JavaScript with no build step, plus a day-by-day simulation and a limited fleet.

Libraries. cpp-httplib, nlohmann/json, Cytoscape.js.

System overview (diagrams (a)–(g) below)

1. The clock moves forward one day.

2. A home is due once the days since its last collection reach capacity / fill rate (1–14 days); the most overdue comes first (c).

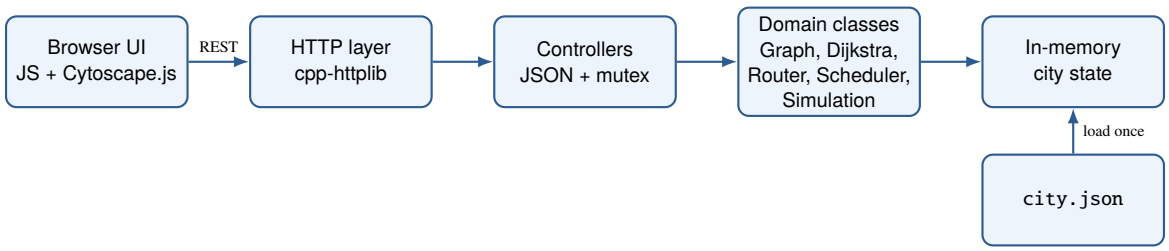
3. Due bins are emptied, hazardous homes go to the hazard trucks (f), and normal homes are ordered and packed onto trucks before routing (e).

4. Each truck chains shortest paths in its list order:

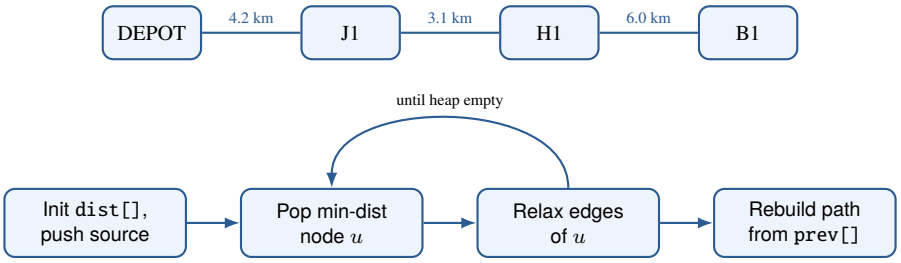
```
current = DEPOT;  visited = { }
for each home in order:
  skip it if in visited
  path = Dijkstra(current → home);  if none, mark unreachable
  empty later stops on the path, add them to visited        (en-route pickup, (d))
  empty home, add to visited;  current = home
then visit the nearest bin for each waste type collected
```

5. Served homes get "last collected = today"; unreachable ones are reported as failed, and carried-over homes rank higher tomorrow.

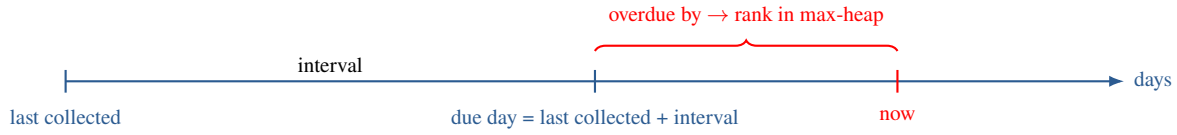
(a) System Pipeline



(b) Graph Storage and Dijkstra

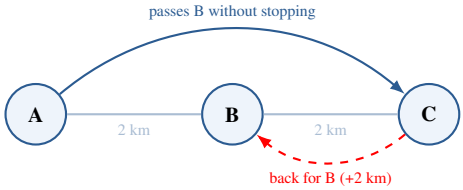


(c) Fill-Rate Scheduling

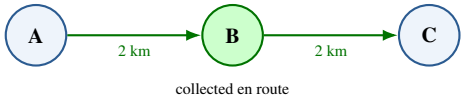


(d) En-Route Pickup (new)

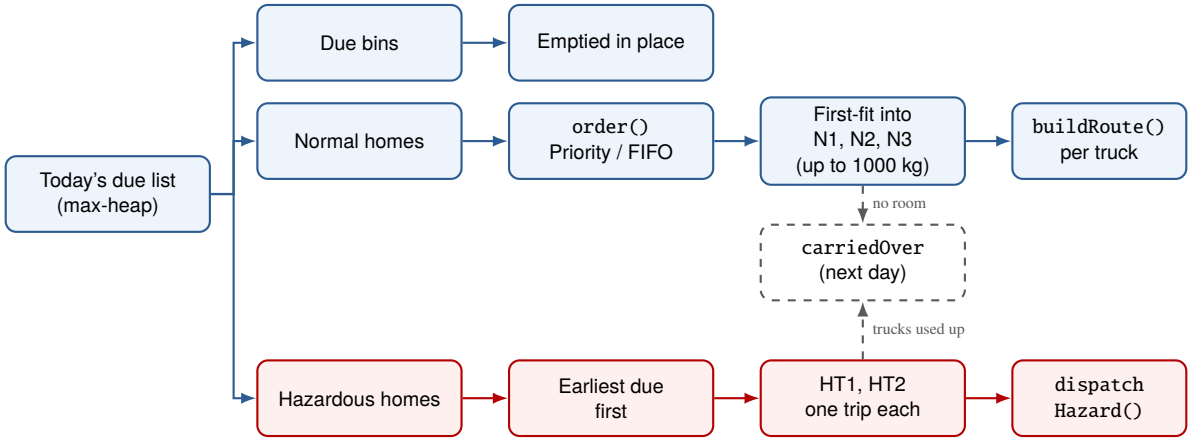
Literal order A → C → B: 6 km



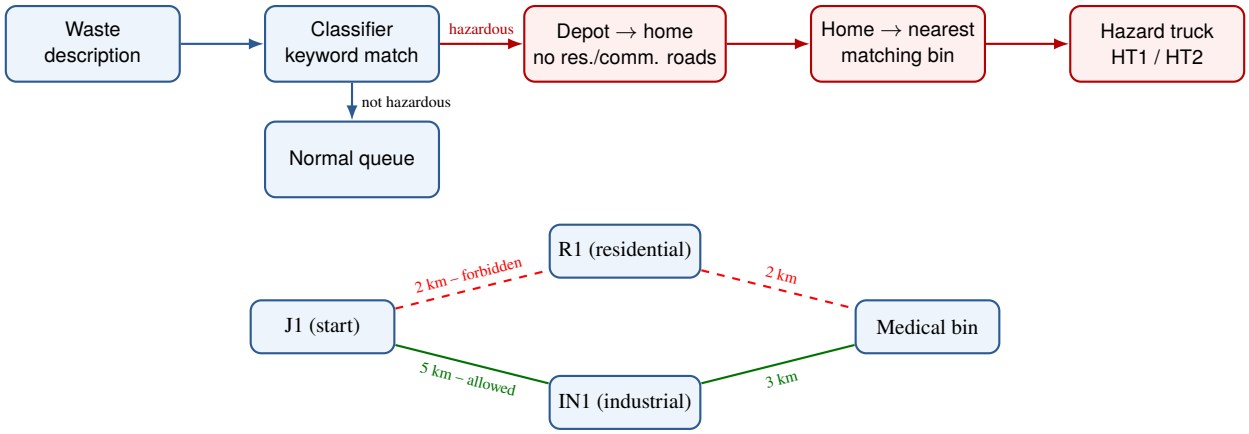
EcoGraph: A → B → C: 4 km



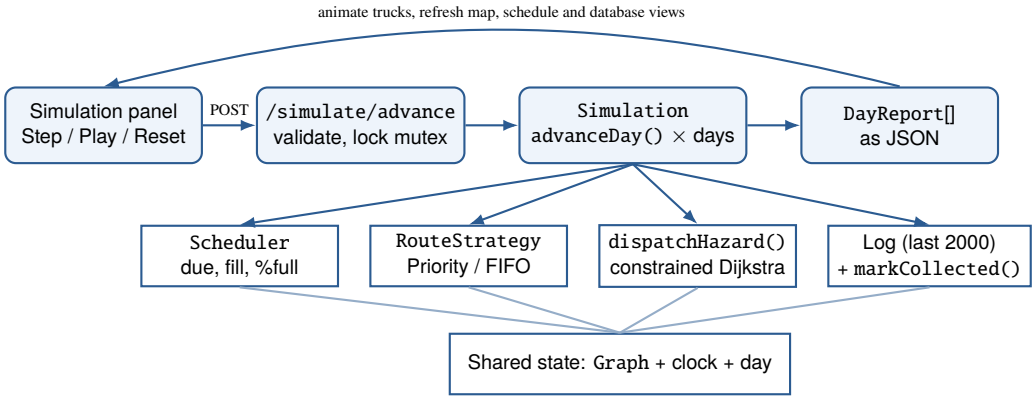
(e) Truck Fleet and Capacity (new)



(f) Automatic Hazard Dispatch and Constrained Dijkstra (new)



(g) Simulation Architecture (new)



Tasks Completed

Task Completed	Team Member
Weighted, zone-tagged city graph stored as an adjacency list, and a JSON loader that reads the Dehradun map (51 nodes, 69 roads) at startup	Tejas Jain (Lead)
HTTP server on cpp-httplib that serves the front end and 13 REST endpoints, with one shared error format and a mutex around the shared state	Tejas Jain
Plain Dijkstra and zone-constrained Dijkstra with path rebuilding, used for normal and hazardous routes respectively	Tejas Jain
Priority and FIFO route builders sharing one template method, with en-route pickup and unloading at the nearest matching bin	Tejas Jain
Keyword-based waste classifier and automatic hazard dispatch to the nearest medical or chemical bin, on industrial and arterial roads only	Tejas Jain
Fill-rate scheduler (interval, due, overdue, percent full) with a max-heap that ranks due locations by how overdue they are	Tejas Jain
Day-by-day simulation with a limited fleet, truck capacity, carry-over to the next day and a collection log	Tejas Jain, Apoorv Negi, Yash Nigam
Four unit test suites covering every backend module, plus a check of the city data	Yash Nigam, Apoorv Negi, Tejas Jain
Project README with build steps, folder layout and the full API reference	Apoorv Negi, Yash Nigam
Map view in Cytoscape.js with truck route animation, hazard-route styling and a shape legend	Yash Nigam, Tejas Jain, Apoorv Negi
Schedule, simulation and database views and the light/dark theme	Tejas Jain, Yash Nigam
Project report	Apoorv Negi, Yash Nigam
Project presentation	Apoorv Negi, Yash Nigam

Challenges/Roadblocks

Road classes beyond residential and commercial. If roads were tagged only as residential or commercial, the hazard truck would have nothing to drive on, because once both are forbidden every road is ruled out. Real cities also have main roads between localities and industrial belts where disposal sites sit, so we added *arterial* and *industrial* road classes. The hazard truck may use these, so it can cross the city without entering a neighbourhood or a market.

Trucks driving past houses on their own list. At first the router followed the scheduler's order literally: given A, C, B with B on the road from A to C, the truck drove past B, reached C, then turned back for B. We added en-route pickup: each leg's path is checked for later stops, which are emptied on the way and skipped when the list reaches them.

Scheduling that never showed its effect. With unlimited trucks, every due home was served every day, so nothing ever overflowed, and choosing Priority or FIFO only changed the stop order, never who got served. We added a limited fleet with 1000 kg per truck and carry-over to the next day. Since an overflowing home heavier than a whole truck would never fit, an empty truck always accepts its first home.

Shared state across requests. cpp-httplib handles requests on several threads, and routing, collection and the simulation all change the same graph, so all shared state sits behind one mutex.

Compiler on Windows. The old MinGW g++ could not build cpp-httplib, which needs proper thread support, so we moved to GCC 13.

Tasks Pending

Task Pending	Team Member (to complete the task)
Write a multi-stage Dockerfile: the first stage compiles the C++17 server with GCC, the second copies only the binary, `city.json` and the front-end folder into a slim runtime image	Tejas Jain
Add a `.dockerignore` that keeps build outputs, report files and the git history out of the image, so it stays small and uploads quickly	Apoorv Negi, Yash Nigam
Choose a host that can run one long-lived container (such as Render, Railway or Fly.io), since the server keeps the city and simulation state in memory	Tejas Jain
Push the image to a container registry and deploy it, with the host's health check pointed at the existing `/health` endpoint	Tejas Jain
Set the `PORT`, `CITY_FILE` and `FRONTEND_DIR` environment variables on the host. The server already reads these, so no code changes are needed	Apoorv Negi, Yash Nigam
Serve the front end from the same origin as the API, leaving `ECO_API_BASE` empty in `config.js`. If it later moves to Vercel, point that value at the API and set `CORS_ORIGIN`	Tejas Jain
Connect the host to the GitHub `main` branch so every pushed commit redeploys automatically	Tejas Jain
Add the live demo link and a one-line `docker run` command to the README	Apoorv Negi, Yash Nigam

Project Outcome/Deliverables

By the end of the project we will hand over:

- A C++17 server that loads a city graph and answers routing, hazard dispatch, classification, scheduling and simulation requests over a REST/JSON API.
- A browser front end that draws the Dehradun map, animates each truck's route, highlights hazardous trips, lists what's due today, and lets you run the city forward one day at a time.
- Unit tests for every core module, plus a sample city file someone else can modify.
- A hosted demo link, the README and API docs, and the Phase-II and Phase-III reports and presentations.

The real outcome we care about is simple: given a list of homes, the system should produce a shorter route than a fixed one would, never send hazardous waste through a residential or commercial street, and stop sending trucks to bins that are nowhere near full.

Progress Overview

We would put the project at roughly 80% done. Everything we promised for Phase-II is working: the graph model, both versions of Dijkstra, the two route builders, the waste classifier, the fill-rate scheduler and the REST API, all with unit tests.

The front end is up as well: the map, route animation, schedule view and database view all work. On top of that we built a day-by-day simulation and a limited truck fleet, because a single route on a static map didn't show whether the scheduling actually worked.

We are on schedule: the front end and the back end are both complete. What is left is a fully deployed version of the system, along with further improvements to the front end and some code cleanup.

Codebase Information

Repository: `github.com/Space-Fighter/EcoGraph` **Branch:** `main`

Layout: `server/` (C++ source, headers, tests, `data/city.json`) and `frontend/` (HTML, CSS, JS modules).

Key files

File	What it does
`server/src/main.cpp`	Starts the server; reads the port, city file and front-end folder
`server/src/controllers.cpp`	The 13 REST endpoints: parses JSON, calls the domain classes, guards shared state with a mutex
`server/src/graph.cpp`	City graph: nodes and zone-tagged roads stored as an adjacency list
`server/src/dijkstra.cpp`	Plain and zone-constrained shortest path
`server/src/scheduler.cpp`	Collection interval, due, overdue and % full; max-heap of due locations
`server/src/waste_classifier.cpp`	Keyword rules: medical, chemical, recyclable, compost (else general)
`server/src/router.cpp`	Priority and FIFO route builders, en-route pickup, nearest-bin unloading, hazard dispatch
`server/src/simulation.cpp`	Advances days, loads trucks, runs routes and hazard trips, keeps the collection log
`server/src/city_loader.cpp`	Reads `city.json` into the graph
`server/data/city.json`	The Dehradun city: localities, nodes, roads, zones, fill rates
`server/tests/`	Unit test programs (Dijkstra, domain, simulation, city data)
`frontend/js/graph-view.js`	Cytoscape.js map and truck route animation
`frontend/js/schedule-view.js`, `simulation-view.js`, `database-view.js`	Schedule, simulation and database panels
`frontend/js/api.js`	Thin `fetch` wrapper around the REST API

Important commits and the modules they added

Commit	Modules / checkpoint
Graph model and Dijkstra	`graph`, `dijkstra` (plain and constrained), Dijkstra tests
REST API server, city data and front end	`scheduler`, `waste_classifier`, `router` (Priority / FIFO), `controllers`, map view
Day-by-day simulation and database view	`simulation`, database view, simulation tests
Hazard truck dispatch and limited fleet	automatic hazard trucks, truck count and capacity in `simulation` and `router`
City mapped to Dehradun (km/kg units)	real coordinates and localities
Full README with API reference	README, API docs
Schematic locality layout	cleaner map layout of the localities
En-route pickup (latest)	en-route collection in `router`

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Unit: Dijkstra (`test_dijkstra`)	Pass	Hand-solved graphs; constrained version never uses a forbidden edge; unreachable target handled
Unit: domain (`test_domain`)	Pass	Classifier rules, scheduler intervals clamped to 1–14 days, Priority vs FIFO ordering and tie-breaks, hazard split
Unit: simulation (`test_simulation`)	Pass	Truck capacity limits, carry-over to next day, one hazard trip per truck per day
Data: city file (`test_city_data`)	Pass	Every home reachable, every hazard home has a legal route to its bin
Manual API checks (curl and browser)	Pass	`/health`, `/classify`, `/schedule`, `/route`, `/simulate/advance` return the expected JSON; bad input gets a clean error

Deliverables Progress

- C++ backend: routing, hazard dispatch, classifier, scheduler, simulation, REST API – **Completed**
- Browser front end with map, route animation, schedule and database views – **Completed** (polish **in progress**)
- Dehradun city data – **Completed**
- Unit tests (4 suites + city data check) – **Completed**
- README and API documentation – **Completed**
- Hosted demo (Dockerfile + deployment) – **Pending**
- Phase-II report – **Completed**; Phase-III report, presentation and demo – **Pending**